Лабораторна робота № 6-7

ТЕМА: РОЗРОБКА ФРОНТЕНДУ ПРОГРАМНОГО ПРОДУКТУ

Об’єкт розробки: мобільний застосунок обліку особистих фінансів «Фінанси» (Finance Tracker), клієнтська частина на фреймворку Flutter (мова Dart).

Мета роботи: здобути практичні навички проєктування та розробки користувацького інтерфейсу (UI/UX) мобільного застосунку, реалізації навігації та інтерактивної взаємодії з користувачем, а також застосування сучасних кросплатформних технологій і підходів до розробки клієнтської частини.

Теоретичні відомості

Розробка фронтенду охоплює створення клієнтської частини програмного продукту, з якою безпосередньо взаємодіє користувач.

Сукупність засобів для обробки та відображення інформації, найбільше пристосованих для зручності користувача. В інтерфейсі мобільного застосунку це екрани, навігація між ними, кольори, розміри й видимість елементів, шрифти, повідомлення та стан елементів (активний, неактивний, заблокований).

Frontend відповідає за:

- відображення інформації;
- навігацію;
- введення та редагування даних;
- взаємодію користувача із системою;
- валідацію введених даних;
- відображення результатів виконання операцій;
- оброблення подій користувача;
- взаємодію з джерелами даних: локальною базою даних і зовнішнім API.

Сучасні мобільні застосунки з декларативним інтерфейсом (Flutter, React Native, SwiftUI) будуються на компонентній архітектурі, що дозволяє повторно використовувати елементи інтерфейсу та керувати станом застосунку (State Management). У Flutter інтерфейс є деревом віджетів (widgets), а стан може передаватися через пакет Provider на основі класів *ChangeNotifier*.

Ключові етапи розробки фронтенду

1. Проєктування UI/UX: створення прототипів (wireframe, mockup).

UI (User Interface) – це сукупність елементів, за допомогою яких користувач взаємодіє з програмною системою. До UI належать:

- кнопки, плаваюча кнопка дії (FloatingActionButton);
- нижня навігація;
- поля введення, перемикачі, чіпи вибору категорій;
- списки операцій;
- форми та нижні панелі (bottom sheets);
- піктограми та кольори;
- шрифти, повідомлення (SnackBar, діалоги), діаграми;
- анімації та переходи.

UX (User Experience) – це загальне враження, яке користувач отримує під час взаємодії із програмним продуктом. UX охоплює значно більше, ніж зовнішній вигляд.

Наприклад, користувач хоче додати витрату в гривнях, доларах або євро. Якщо він:

1. швидко знаходить кнопку «Операція»;
2. розуміє, які дані потрібно ввести;
3. отримує зрозумілі підказки, якщо дані неповні;
4. успішно зберігає операцію;
5. бачить оновлений баланс і розуміє, що робити далі, —

застосунок має хороший UX.

Usability (зручність використання) – характеристика програмної системи, яка визначає, наскільки легко користувач може навчитися працювати із системою та виконувати необхідні завдання.

Для оцінювання usability можна використовувати такі характеристики: зрозумілість; простоту навчання; ефективність; запам’ятовуваність; кількість помилок; задоволеність користувача.

Основні принципи usability:

- **Простота** – інтерфейс не повинен містити зайвих елементів.
- **Послідовність** – однакові елементи повинні працювати однаково в усьому застосунку.
- **Зрозумілість** – користувач повинен розуміти призначення елемента без додаткових пояснень.

- **Видимість стану системи** – застосунок повинен повідомляти користувачу, що відбувається (наприклад, прогрес-бар бюджету, повідомлення про перевищення ліміту).
- **Запобігання помилкам** – хороший інтерфейс не лише повідомляє про помилку, а й допомагає її уникнути. Наприклад, кнопка збереження неактивна, доки дані форми некоректні.
- **Інформативні повідомлення про помилки**.
- **Контроль користувача** – користувач повинен розуміти, що відбувається, і мати можливість скасувати дію там, де це доречно.

Приклад інформативного повідомлення:

Погано:

```
Invalid input
```

Краще:

```
Введіть суму більше нуля.
```

Приклад контролю користувача (перед видаленням операції):

```
Видалити операцію?
[Скасувати]  [Видалити]
```

Wireframe

Wireframe – схематичне представлення майбутнього інтерфейсу, яке дозволяє перевірити структуру екранів до початку повноцінної розробки. Wireframe показує розташування елементів, навігацію, блоки інформації, кнопки та поля. Оформлення при цьому максимально просте.

Prototype

Prototype (прототип) – модель майбутнього програмного продукту, яка дозволяє перевірити його функціональність, навігацію та взаємодію з користувачем. Prototype може бути статичним або інтерактивним, різного рівня деталізації.

Адаптивний дизайн для мобільних пристроїв

Адаптивний дизайн – підхід до створення інтерфейсів, за якого застосунок коректно відображається на різних розмірах екрана та з різними налаштуваннями системи. Основні принципи:

- гнучка ширина блоків (Expanded, Flexible, Wrap);

- безпечні зони екрана (SafeArea) та врахування вирізів і індикатора Home;
- поведінка при появі екранної клавіатури (нижні панелі піднімаються над клавіатурою);
- масштабування довгих значень (FittedBox);
- світла і темна теми оформлення;
- розташування елементів, зручне для керування великим пальцем;
- нижня навігація замість бічного меню.

Порядок виконання роботи

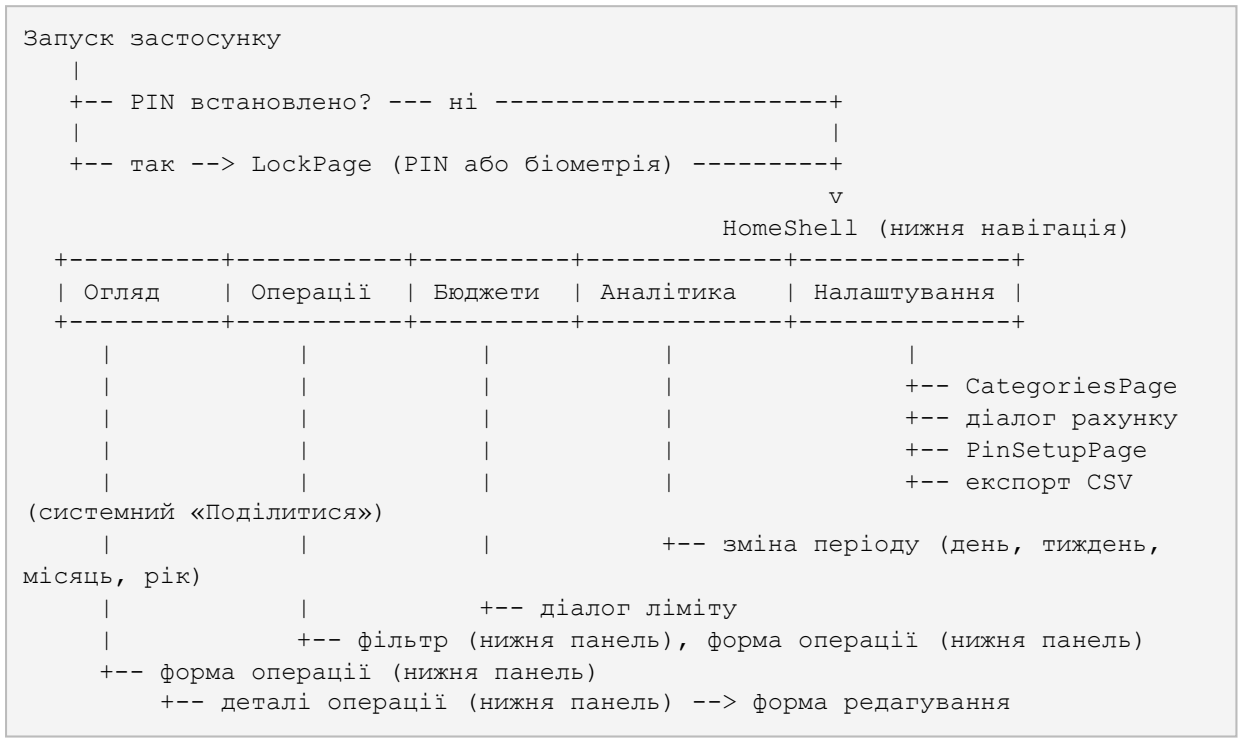
1. Визначити основні користувацькі сценарії: огляд балансу, додавання операції у трьох валютах, перегляд деталей операції, місячні ліміти бюджету, аналітика за періодами, захист PIN-кодом.
2. Визначити необхідні екрани: п’ять основних розділів, екран блокування, форма операції, деталі операції, категорії, діалоги рахунку і ліміту.
3. Створити wireframe основного екрана (огляд), екрана зі списком даних (операції) та форми додавання/редагування.
4. Розробити prototype основних переходів: огляд → деталі операції → форма редагування; перемикання між розділами нижньої навігації.
5. Ініціалізувати проєкт командою *flutter create* і підключити пакети у файлі *pubspec.yaml*: provider, sqflite, path, path_provider, fl_chart, intl, crypto, flutter_secure_storage, local_auth, shared_preferences, csv, share_plus.
6. Декомпозувати UI на окремі віджети (картка курсів, картка статистики, плитка операції, прогрес-бар бюджету, форма операції, деталі операції, панель PIN-коду).
7. Реалізувати адаптивну верстку для мобільних пристроїв (iOS і Android) з використанням Material 3, безпечних зон, підтримки клавіатури та світлої/ темної теми.
8. Реалізувати взаємодію з користувачем: обробка подій (натискання, свайп, вибір у чіпах і списках), валідація даних у формі операції з підказкою під кнопкою збереження.
9. Реалізувати асинхронні запити: завантаження офіційних курсів НБУ через REST-сервіс з тайм-аутами, обробкою помилок і кешуванням; операції CRUD виконувати асинхронно через локальну базу SQLite.
10. Провести аналіз usability запропонованого інтерфейсу.
11. Обґрунтувати прийняті проєктні рішення.

Звіт повинен містити:

- карту навігації;
- схему декомпозиції інтерфейсу;
- макети основних екранів;
- структуру проєкту;
- код основних компонентів із коментарями щодо роботи стану та викликів API;
- скріншоти готового інтерфейсу на мобільному пристрої (iOS-симулятор) у світлій і темній темах;
- аналіз usability та висновки.

Розділ 1. Карта навігації

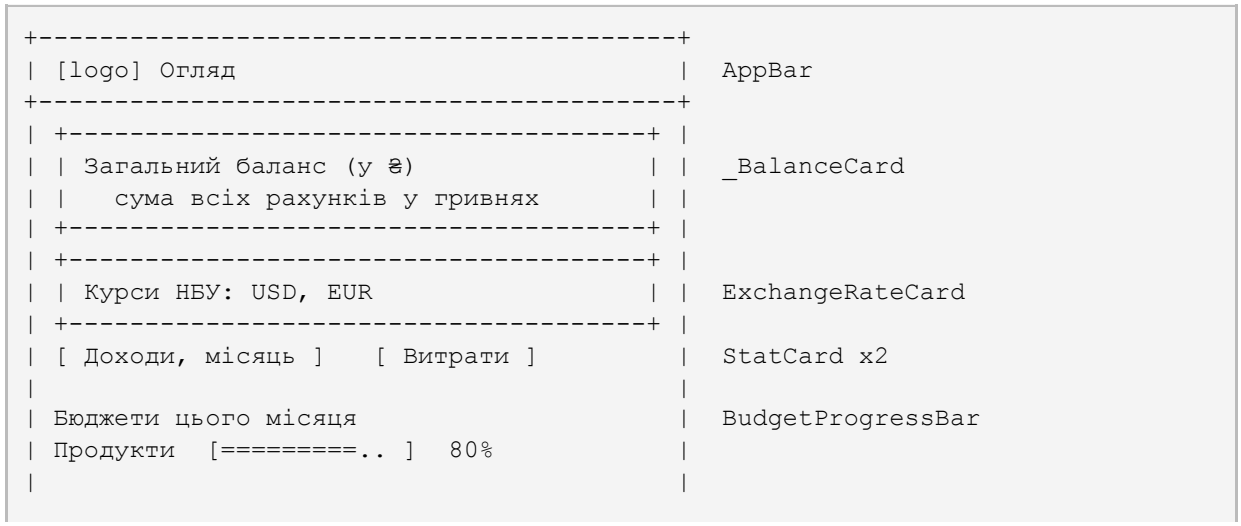
Після запуску застосунок перевіряє, чи встановлено PIN-код. Якщо так, спочатку відкривається екран блокування (*LockPage*), після успішного входу — основна оболонка з нижньою навігацією (*HomeShell*) з п’ятьма розділами. Розділи зберігають свій стан під час перемикання.



Застосунок блокується, коли переходить у фон (стан *AppLifecycleState.paused*). Після повернення потрібно знову ввести PIN-код або підтвердити біометрію.

Розділ 2. Макети основних екранів

2.1. Макет екрана «Огляд» (wireframe)



```
| Останні операції                          |  TransactionTile (тап -> деталі)
| (о) Продукти            -350,00 ₴         |
|                                  [+ Операція]  FloatingActionButton
+-------------------------------------------+
| Огляд | Операції | Бюджети | Аналітика | Налаштування |  нижня навігація
+-------------------------------------------+
```

2.2. Макет форми операції (wireframe)

```
+-----------------------------------------+
|  ——  (ручка нижньої панелі)             |
|  Нова операція                          |
|  [ Витрата | Дохід ]                    |  SegmentedButton
|  Валюта: [ ₴ | $ | € ]                   |  валюта операції
|  Сума: [ 0,00            ]              |  поле введення (лише числа)
|  Категорія: (Продукти)(Транспорт)...    |  чіпи вибору
|  Рахунок:  [ Картка        v ]          |  вибір рахунку
|  Дата: [ 18.10.2026 ]  Час: [ 14:30 ]   |
|  Примітка: [                     ]      |  до 200 символів
|                                         |
|  Підказка: «Оберіть категорію»          |  показується, доки дані неповні
|              [ Зберегти ]               |  неактивна, доки є підказка
+-----------------------------------------+
```

Розділ 3. Схема декомпозиції інтерфейсу

Інтерфейс декомпозовано на віджети. Кожен віджет відповідає за одну частину екрана. Дані передаються через параметри, а зміни стану отримуються через Provider (*context.watch*, *context.read*).

```
FinanceApp  (app.dart: MaterialApp, локаль uk, світла/темна тема)
 |
 +-- LockPage (lock_page.dart)          якщо PIN встановлено і застосунок
заблоковано
 |    +-- PinPad (pin_pad.dart)
 |
 +-- HomeShell (home_shell.dart)        IndexedStack + нижня навігація
      +-- DashboardPage (dashboard_page.dart)
      |    +-- _BalanceCard (приватний віджет сторінки)
      |    +-- ExchangeRateCard (exchange_rate_card.dart)
      |    +-- StatCard (stat_card.dart)
      |    +-- BudgetProgressBar (budget_progress_bar.dart)
      |    +-- TransactionTile (transaction_tile.dart)
      |          +-- showTransactionDetails (transaction_details_sheet.dart)
      |                +-- форма операції (transaction_form_sheet.dart)
      +-- TransactionsPage (transactions_page.dart)
      |    +-- пошук, _FilterSheet (фільтр)
      |    +-- TransactionTile у Dismissible, підтвердження через
confirmAction
      +-- BudgetsPage (budgets_page.dart)
      |    +-- BudgetProgressBar, _BudgetDialog (ліміт), confirmAction
      +-- AnalyticsPage (analytics_page.dart)
      |    +-- перемикач періоду, StatCard
      |    +-- діаграми PieChart і LineChart (fl_chart)
      +-- SettingsPage (settings_page.dart)
           +-- PIN-код (PinSetupPage, confirmAction при вимкненні)
           +-- CategoriesPage, _AccountDialog, експорт CSV
```

```
Спільні компоненти:
  common_widgets.dart: SectionHeader, EmptyNotice, confirmAction
  pin_pad.dart, stat_card.dart, exchange_rate_card.dart, transaction_tile.dart
```

Розділ 4. Структура проєкту

```
finance_tracker/
├── android/, ios/                          нативні проєкти Android і iOS
├── assets/images/logo.png                  логотип у AppBar
├── lib/
│   ├── main.dart                           точка входу, створення ViewModel,
MultiProvider
│   ├── app.dart                            MaterialApp, локаль uk, вибір
LockPage або HomeShell
│   ├── core/
│   │   ├── constants/
│   │   │   ├── currency.dart               валюти (UAH, USD, EUR)
│   │   │   └── palette.dart                кольори категорій
│   │   ├── theme/app_theme.dart            світла і темна теми (Material 3)
│   │   └── utils/formatters.dart           форматування сум і дат
│   ├── models/models.dart                  Account, FinanceCategory,
FinanceTransaction, Budget
│   ├── services/
│   │   ├── database_helper.dart            відкриття SQLite, схема, міграції
│   │   ├── finance_repository.dart         доступ до даних (CRUD і вибірки)
│   │   ├── exchange_rate_service.dart      курси НБУ, кеш у SharedPreferences
│   │   ├── security_service.dart           PIN-код (хеш) і біометрія
│   │   └── csv_exporter.dart               експорт операцій у CSV
│   ├── logic/
│   │   ├── finance_view_model.dart         стан операцій, рахунків, категорій,
бюджетів
│   │   ├── exchange_rate_view_model.dart   стан курсів НБУ
│   │   ├── security_view_model.dart        стан блокування і PIN-коду
│   │   ├── app_settings_view_model.dart    налаштування (тема)
│   │   ├── analytics_calculator.dart       розрахунок аналітики за періодами
│   │   ├── budget_validator.dart           рівні лімітів (ok, warning, exceeded)
│   │   └── transaction_filter.dart         критерії пошуку і фільтрації
│   └── ui/
│       ├── pages/                          home_shell, dashboard_page,
transactions_page,
│       │                                   budgets_page, analytics_page,
settings_page,
│       │                                   categories_page, lock_page,
pin_setup_page,
│       │                                   transaction_form_sheet
│       └── components/                     transaction_tile,
transaction_details_sheet,
│                                           budget_progress_bar,
exchange_rate_card, stat_card,
│                                           common_widgets, pin_pad
└── test/widget_test.dart                   widget-тести
```

Розділ 5. Код основних компонентів

5.1. Ініціалізація та розподіл стану (main.dart)

Базу даних, курси і налаштування ініціалізовано до запуску інтерфейсу. ViewModel передаються через *MultiProvider*, тому віджети отримують дані без передачі через конструктори. Курси НБУ завантажуються у фоні (*unawaited*), щоб не затримувати запуск. Кожне оновлення курсів передає нові значення в *FinanceViewModel* для перерахунку сум.

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await initializeDateFormatting('uk');

  final prefs = await SharedPreferences.getInstance();
  final security = SecurityViewModel(SecurityService());
  await security.init();

  final finance =
FinanceViewModel(SqliteFinanceRepository(DatabaseHelper.instance));
  final exchange = ExchangeRateViewModel(ExchangeRateService(prefs));
  finance.setRates(exchange.rates);
  exchange.addListener(() => finance.setRates(exchange.rates));
  await finance.load();
  unawaited(exchange.refresh());

  final settings = AppSettingsViewModel(prefs);

  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider.value(value: security),
        ChangeNotifierProvider.value(value: settings),
        ChangeNotifierProvider.value(value: finance),
        ChangeNotifierProvider.value(value: exchange),
      ],
      child: const FinanceApp(),
    ),
  );
}
```

5.2. Виклик API НБУ та кешування (exchange_rate_service.dart)

Запит виконується асинхронно з тайм-аутами: 8 с на з'єднання і 10 с на відповідь. Успішна відповідь зберігається в *SharedPreferences*, тому застосунок працює з останніми збереженими курсами, коли мережі немає. Неочікувана відповідь викликає виняток, який обробляє ViewModel. Клієнт HTTP закривається у блоці *finally*.

```
/// Downloads the current NBU rates and stores them for offline use.
Future<ExchangeRates> fetch() async {
  final client = HttpClient()..connectionTimeout = const Duration(seconds: 8);
  try {
    final request = await client.getUrl(_endpoint);
    final response = await request.close().timeout(const Duration(seconds:
10));
    if (response.statusCode != HttpStatus.ok) {
      throw HttpException('НБУ відповів статусом ${response.statusCode}', uri:
```

```
_endpoint);
    }
    final body = await response.transform(utf8.decoder).join();
    final rates = ExchangeRates.fromNbuJson(jsonDecode(body) as
List<dynamic>);
    if (rates.isEmpty) {
      throw const FormatException('У відповіді немає курсів USD або EUR');
    }
    await _prefs.setString(
      _cacheKey,
      jsonEncode({'date': rates.date, 'rates': rates.toUah}),
    );
    return rates;
  } finally {
    client.close();
  }
}
```

5.3. Конвертація валют через базову валюту (exchange_rate_service.dart)

Курси НБУ зберігаються як «гривень за одиницю валюти». Конвертація між двома валютами виконується через гривню. Якщо курсу будь-якої з валют немає, метод повертає *null*, а форма операції блокує збереження.

```
double? convert(double amount, {required String from, required String to}) {
  if (from == to) return amount;
  final fromRate = from == Currencies.base ? 1.0 : toUah[from];
  final toRate = to == Currencies.base ? 1.0 : toUah[to];
  if (fromRate == null || toRate == null) return null;
  return amount * fromRate / toRate;
}
```

5.4. Валідація форми операції (transaction_form_sheet.dart)

Фрагмент методу *build()*. Підказка формується з першої невиконаної умови і відображається під кнопкою збереження. Прапорець *_saving* запобігає повторному збереженню під час асинхронного запису в базу.

```
final amount = _parsedAmount;
final hint = amount == null || amount <= 0
    ? 'Введіть суму більше нуля'
    : categoryId == null
        ? 'Оберіть категорію'
        : accountId == null
            ? 'Оберіть рахунок'
            : rates.convert(amount, from: currency, to: accountCurrency) ==
null
                ? 'Немає курсу для $currency. Спробуйте оновити курси'
                : null;
final canSave = hint == null && !_saving;
```

5.5. Визначення рівня ліміту бюджету (budget_validator.dart)

Рівень визначається за часткою витрат від ліміту. Поріг попередження — 80%, перевищення — 100%.

```
static const warningThreshold = 0.8;

final ratio = budget.limitAmount == 0 ? 0.0 : spent / budget.limitAmount;
final level = ratio >= 1
    ? BudgetLevel.exceeded
    : ratio >= warningThreshold
        ? BudgetLevel.warning
        : BudgetLevel.ok;
```

Розділ 6. Скріншоти готового інтерфейсу

[Вставити скріншоти з iOS-симулятора: екран «Огляд» (світла тема), екран «Огляд» (темна тема), форма операції, деталі операції, екран «Аналітика»]

Розділ 7. Аналіз usability

Критерій	**Реалізація у застосунку**
Платформи та адаптивність	Застосунок працює на iOS і Android. Інтерфейс побудовано на Material 3. Нижні панелі (форма, деталі, фільтр) враховують висоту клавіатури і безпечні зони екрана. Довгі суми в балансі масштабуються через *FittedBox*.
Тема оформлення	Системна, світла або темна тема. Вибір доступний у розділі «Налаштування».
Навігація	Нижня панель з п’ятьма розділами і підписами. Кнопки мають *Tooltip* та семантичні мітки для допоміжних технологій.
Запобігання помилкам	Кнопка збереження неактивна, доки дані некоректні. Причина показується в підказці під кнопкою. Сума має бути більшою за нуль. Для обраної валюти має бути відомий курс.
Підтвердження дій	Видалення операції, видалення ліміту і вимкнення PIN-коду потребують підтвердження (діалог *confirmAction*).
Порожні стани	Якщо даних немає, застосунок показує пояснення через компонент *EmptyNotice*.
Робота без інтернету	Операції зберігаються в локальній базі SQLite. Останні курси НБУ кешуються і використовуються, коли мережі немає.
Попередження про бюджет	Після збереження операції, що перетнула поріг 80% або 100% ліміту, з’являється повідомлення (SnackBar) з відповідним текстом.

Безпека даних	PIN-код зберігається у вигляді хешу SHA-256 із сіллю в захищеному сховищі (*flutter_secure_storage*). Порівняння хешів виконується за постійний час. Застосунок блокується при переході у фон.

Розділ 8. Обґрунтування вибору технологій

Технологія	**Призначення**	**Обґрунтування**
Flutter (Dart), Material 3	Клієнтська частина для iOS і Android	Один код для двох платформ; готові компоненти Material 3 для нижньої навігації, форм і діалогів
Provider (ChangeNotifier)	Керування станом	Простий і зрозумілий підхід для застосунку такого розміру; стан винесено в окремі ViewModel-класи (розділ 4)
sqflite (SQLite)	Локальне зберігання даних	Працює без мережі; схема відповідає моделі 3NF з лабораторної роботи № 5; зміни схеми через міграції *onUpgrade*
НБУ (bank.gov.ua), HttpClient	Офіційні курси USD і EUR до UAH	Офіційне джерело курсів; запит без ключа API і лише для читання; результат кешується
SharedPreferences	Кеш курсів НБУ і налаштування	Просте сховище пар «ключ–значення» для невеликих даних
flutter_secure_storage, crypto	Зберігання PIN-коду	Захищене сховище платформи; хеш SHA-256 із сіллю замість відкритого PIN-коду
local_auth	Біометрія	Вхід через Face ID або відбиток пальця засобами системи
fl_chart	Діаграми аналітики	Кругова і лінійна діаграми для структури витрат і динаміки
intl, локаль uk	Українське форматування дат і сум	Коректні назви місяців і дні тижня українською мовою
share_plus, csv	Експорт операцій	Файл CSV відкривається в табличному редакторі через системне меню «Поділитися»

Застосунок не використовує веб-фреймворки і власний сервер. Це персональний облік даних одного користувача, тому дані зберігаються на

пристрої. Єдине зовнішнє звернення — запит до публічного API НБУ лише для читання курсів.

Висновки

У ході лабораторних робіт № 6–7 реалізовано клієнтську частину мобільного застосунку «Фінанси» на Flutter. Застосунок містить п’ять основних розділів: огляд, операції, бюджети, аналітику і налаштування. Реалізовано додавання, редагування і видалення операцій у трьох валютах з конвертацією за курсами НБУ, перегляд деталей операції, пошук і фільтрацію, місячні ліміти з попередженнями, аналітику за періодами з діаграмами, експорт у CSV, захист PIN-кодом і біометрією, а також вибір теми оформлення.

Застосунок зібрано і запущено на симуляторі iOS. Тести (14 тестів) виконуються успішно.

Обмеження поточної версії: дані зберігаються лише на пристрої, синхронізація між пристроями і хмарне резервне копіювання не реалізовані. Перспективою розвитку є синхронізація з сервером, резервне копіювання та розширення аналітики.